ELI — Finetuning Guide

ELI v2.1.66

August 2026

Contents

ELI Fine-Tuning Guide — Train a Model the Correct Way (End-to-End)

A meticulous, do-this-exactly walkthrough for turning **a base model you choose** into an **ELI-native GGUF**, then loading it back into ELI. Covers the *why* as well as the *how*, so it's a guide you can reason from, not cargo-cult.

Method: QLoRA (4-bit base + LoRA adapters). **Pipeline:** the scripts in `training/` (extract → train → merge → convert → install). **Default base:** `Qwen/Qwen3-8B` — but the whole guide is written around a single `BASE=...` you set once (Stage 0), so the *same commands* train **any** model you pick into a GGUF.

> Scope: this is **fine-tuning** (adapting a pretrained model), **not** pretraining from scratch (that needs thousands of GPU-hours and is out of reach here — and unnecessary).
>
> Hardware: QLoRA keeps this on a single consumer GPU. The defaults target an ~8 GB card; the `--gpu-mem` / base-size knobs in Stage 2 scale it up or down for whatever you have. No specific GPU is assumed anywhere.

---

0. The five principles (read before touching anything)

1. **Persona stays dynamic — never bake it into the weights.** ELI's persona is live (`persona_updater` + overlay + the per-turn memory brief). You fine-tune the **stable layer only**: ELI's *voice / manner* and its *behavioral contracts* (No-Fake-Actions, grounding, dry register). You do **not** train in user facts, memories, dates, or a fixed persona prompt. The old `models/extract_training_data*.py` dumped `memories[-50:]` in as system prompts — that froze state. The new `extract_eli_dataset.py` deliberately does not.

2. **LoRA, not full fine-tune.** A full fine-tune of an 8B model needs ~60–100 GB VRAM. LoRA trains tiny low-rank adapters (<1% of params) on top of a frozen 4-bit base → fits 8 GB. For a *style/voice* shift (what we want), LoRA is not a compromise — it's the right tool.
3. **Data quality □ data quantity.** A few hundred clean, on-voice examples beat thousands of noisy ones. Garbage in → garbage (or overfit) out.
4. **Match the chat template to the target model.** Qwen3 uses ChatML. The dataset MUST be formatted with the *target model's* tokenizer template, or the model learns the wrong control tokens and produces garbage. (`extract_eli_dataset.py` does this via `tokenizer.apply_chat_template`.)
5. **The base model must have enough context for ELI's brief.** ELI's runtime prompt is ~7–9k tokens. A 4k-context base (phi-3-mini) **cannot** run ELI — it produced literal garbage (`_....,chknce/you,//`). Qwen3-8B's 40,960 context is why it's the pick. (This is enforced at load by the ctx tuner — see §7.)

---

1. Concepts you must understand (so this is rigorous, not ritual)

- **Base vs Instruct weights:** you fine-tune **HF weights** (a directory of `.safetensors` + tokenizer), *not* a GGUF. GGUF is the **inference** format ELI loads; HF is the **training** format. They are not interchangeable — you'll download HF for training and produce a GGUF at the end.
- **LoRA (Low-Rank Adaptation):** freezes the base weights and learns two small matrices per target layer whose product is added to the original weight. `r` = the rank (capacity of the change); `alpha` = a scaling factor (effective LR multiplier ≈ `alpha/r`). Small `r` (8) = cheap, good for style; large `r` = more capacity, more overfit risk.
- **QLoRA (4-bit):** the frozen base is quantized to 4-bit (nf4) to fit VRAM; the LoRA adapters train in fp16. Quality loss from 4-bit base is negligible for fine-tuning.
- **`target_modules`:** which weight matrices get adapters. We adapt all attention + MLP projections (`q/k/v/o_proj`, `gate/up/down_proj`) — the standard full coverage.
- **Quantization (inference):** the merged model is converted to GGUF and quantized — `Q4_K_M` for an 8 GB card (best size/quality balance), `Q5_K_M`/`Q8_0` if you have headroom.
- **Catastrophic forgetting / overfitting:** too many steps or too high `r`/LR makes the model *memorize* your data and *forget* its general ability (and parrot you robotically). Defend with: low `r`, modest LR, **few steps**, and watching the loss (see §4).
- **`n_ctx_train`:** the base model's trained context window. It sets the ceiling for inference ctx; ELI reads it from GGUF metadata at load (the ctx tuner).

---

2. Environment setup (once)

```
# Use the project venv (CUDA already verified there).
cd ~/Desktop/ELI_MKXI-main_MAY_NEWEST
.venv/bin/python -m pip install --upgrade unsloth trl transformers datasets accelerate bitsandbytes hugging

# llama.cpp — needed to convert HF -> GGUF and quantize (if not already present):
#   git clone https://github.com/ggerganov/llama.cpp && cd llama.cpp && make
#   (note its path; you'll point merge/convert at it)
```

Sanity checks:

```
.venv/bin/python -c "import torch; print('CUDA:', torch.cuda.is_available(), torch.cuda.get_device_name(0))'
.venv/bin/python -c "import unsloth, trl, transformers, datasets; print('train stack OK')"
```

> **Hardware honesty:** QLoRA-training an 8B on 8 GB works but is **tight and slow** (heavy CPU offload). If it crawls or OOMs, drop to **Qwen3-4B** as the base (`--base-model Qwen/Qwen3-4B`) — trains far more comfortably, still 32k+ context, still a strong ELI substrate.

---

3. Step 0 — Choose your model, then get its base HF weights

0a. Choose the base (the decision that matters most)

You train **HF weights**, not a GGUF (see §1). Pick a base whose context is **≥16k** (ELI's brief is ~7–9k tokens — a 4k base produces literal garbage). Set it **once** as a shell variable and every command below is identical regardless of which model you chose:

```
export BASE="Qwen/Qwen3-8B"          # default — strong ELI substrate, 40k ctx, fits ~8 GB at Q4
# export BASE="Qwen/Qwen3-4B"        # lighter — trains faster/comfier, still 32k+ ctx
# export BASE="microsoft/phi-4"      # 14B dense (MIT), needs more VRAM/offload
# export BASE="tiiuae/Falcon3-10B-Instruct"   # or any HF instruct model with ≥16k ctx
export NAME="eli-$(basename "$BASE" | tr '[:upper:]' '[:lower:]')"   # output gguf base name
```

Want	Pick	Why
Best all-round default	`Qwen/Qwen3-8B`	40k ctx, reasoning-capable, fits 8 GB at Q4
Lightest / fastest train	`Qwen/Qwen3-4B`	comfortable on 8 GB, still ≥32k ctx
Strong reasoning, more VRAM	`microsoft/phi-4`	14B dense, MIT-licensed
Try something else	any HF instruct repo	**must have `n_ctx_train` ≥ 16k**

> The GGUF keys in ELI's download catalog (`qwen3-8b`, `phi-4`, …) are *inference* builds. For **training** you need the matching **HF** repo (the `BASE` above), not the GGUF.

0b. Download the base HF weights (~16 GB for an 8B; the one online step)

```
.venv/bin/huggingface-cli download "$BASE" --local-dir "training/base/$(basename "$BASE")"
export BASE_LOCAL="training/base/$(basename "$BASE")"   # use this in the commands below
```

You can also pass the HF id directly (`--base-model "$BASE"`) to let HF cache it instead.

---

4. The pipeline (the meticulous part)

Step 1 — Extract the dataset (voice/manner only)

```
.venv/bin/python training/extract_eli_dataset.py \
    --base-model Qwen/Qwen3-8B \
    --out training/datasets/eli_voice.jsonl
```

What it does: reads your stored conversations (`artifacts/conversations/*.json`), keeps the USER↔ELI turns, formats each with **Qwen3's chat template**, and writes `{"text": ...}` JSONL. By default `--system-mode none` and **memory-state injection is excluded** (persona stays dynamic).

Curate before you train (this is where quality is won): - Open the JSONL. Remove turns where ELI was *wrong, off-voice, or fabricating* (you don't want to teach the failures we spent this week fixing). - Keep turns that exemplify the voice you want: dry, direct, honest, no fake actions, grounded. - Aim for **a few hundred good examples**, not thousands of mediocre ones. - Optional: hand-write a dozen "gold" exemplars of perfect ELI behavior (refusing to fake an action, admitting uncertainty, the dark-humor register) and append them — high-signal.

Step 2 — Train the LoRA adapter

Always do a smoke run first (10 steps) to confirm the loop before committing hours:

```
.venv/bin/python training/train_lora.py \
    --base-model Qwen/Qwen3-8B \
    --dataset training/datasets/eli_voice.jsonl \
    --out training/runs/eli-qwen3-8b-lora \
    --max-seq-len 4096 --max-steps 10
```

Then the real run (`--max-steps 200–400`). Hyperparameters, and how to set them:

Flag	Default	What it controls	How to tune
`--max-steps`	300	total optimizer steps	start 200–400; **watch loss** (below). More ≠ better.
`--lora-r`	8	adapter capacity	8 for voice; 16 only if voice isn't "sticking". Higher → overfit.
`--lora-alpha`	16	adapter scaling (≈2×r)	keep ≈ 2×r.
`--lr`	2e-4	learning rate	1e-4 (gentle) … 2e-4 (standard). Too high → loss spikes/garbage.
`--max-seq-len`	4096	max example length	raise to fit your longest multi-turn example; costs VRAM.
`--batch` / `--grad-accum`	1 / 8	effective batch = 8	leave at 1×8 on 8 GB; raise grad-accum for a smoother loss.
`--gpu-mem` / `--cpu-mem`	5.5GiB / 20GiB	VRAM/RAM split	lower `--gpu-mem` if OOM; the rest spills to CPU (slower).

Reading the loss (your only real gauge): - It should **decrease then flatten**. A gentle decline to a plateau ≈ healthy. - **Loss → near 0** = memorizing your data (overfit). Stop earlier / lower r/steps. - **Loss flat/noisy from the start** = LR too low, data too small, or template mismatch. - Save adapter checkpoints; if a later checkpoint sounds *more robotic*, use an earlier one.

Output: a LoRA adapter at `training/runs/eli-qwen3-8b-lora/`.

Step 3 — Merge → convert → quantize → install

```
.venv/bin/python training/merge_and_convert.py \
    --base-model Qwen/Qwen3-8B \
    --adapter training/runs/eli-qwen3-8b-lora \
    --out-name eli-qwen3-8b --quant q4_k_m
```

Produces and installs `models/eli-qwen3-8b-q4_k_m.gguf`. (Manual llama.cpp fallback if Unsloth's GGUF export fails — see `training/README.md`.)

Step 4 — Load it in ELI

Model menu / Auto Detect → pick the new GGUF. The **ctx tuner** reads its real `n_ctx_train` (40960), right-sizes ctx to ~16k, and keeps it on the GPU. The **dynamic persona + memory** are still supplied fresh by the runtime — your fine-tune just made the *substrate* speak ELI.

---

5. Verification — is the fine-tune actually good?

Don't trust "the loss went down." Load it and check **four** things: 1. **Voice:** does it sound like ELI (dry, direct, honest) *without* the heavy persona brief leaning on it? Compare a few prompts vs base Qwen3-8B. 2. **Contracts hold:** ask it to "pause spotify" when it can't, or to analyze a file — it must **not fabricate** an action/result (No-Fake-Actions). Ask a grounded question — it must not confabulate. 3. **Persona still dynamic:** confirm the runtime overlay/memory still steer it (e.g., it picks up a new stated preference this session). If it ignores the live brief and only parrots training data → you overfit; retrain gentler. 4. **No capability regression:** a couple of reasoning/coding prompts should be ~as good as base Qwen3-8B. If markedly worse → overfit; fewer steps / lower `r`. Keep a tiny held-out set of prompts and run it after every training run — that's your eval.

---

6. Troubleshooting (cause → fix)

Symptom	Cause	Fix
Garbage output (`_...,/you//`)	ctx > model's `n_ctx_train`, or wrong chat template	ensure base ≥16k ctx; confirm extract used the *target* tokenizer template
CUDA OOM during training	base too big for VRAM	lower `--gpu-mem`; smaller base (Qwen3-4B); lower `--max-seq-len`; `--batch 1`
Loss won't drop	LR too low / data too small / template mismatch	raise `--lr` to 2e-4; add data; re-check template
Model parrots you, ignores live persona	overfit	fewer `--max-steps`, lower `--lora-r`, earlier checkpoint
Worse at reasoning than base	overfit / too-high rank	lower `r`, fewer steps
GGUF convert fails	Unsloth export quirk	use the manual llama.cpp `convert_hf_to_gguf.py` + `llama-quantize` path (README)
Fine-tune fabricates actions	trained on ELI's *bad* turns	scrub the dataset of fabrication/off-voice turns; add gold no-fake exemplars

---

7. How this ties into ELI's runtime (so the loop is closed)

- **Model-agnostic loader:** drop the produced GGUF in `models/`; ELI finds it, no config.
- **Ctx tuner (shipped):** reads the model's real `n_ctx_train` from metadata, right-sizes ctx, maximises GPU layers, and *warns* if a model's context is too small for ELI's brief — so a bad-fit base can never silently degrade you again.
- **Contracts stay in code:** No-Fake-Actions, grounding, fenced-JSON tolerance, empty-`<think>` recovery are **runtime guards** (this week's commits). The fine-tune *reinforces* the voice; the guards *enforce* the behavior. Belt and braces.

---

8. Quick reference (copy-paste — works for ANY chosen base)

```
# choose your model ONCE (Stage 0)
export BASE="Qwen/Qwen3-8B"                              # ← change this to train a different model
export NAME="eli-$(basename "$BASE" | tr '[:upper:]' '[:lower:]')"
export BASE_LOCAL="training/base/$(basename "$BASE")"
```

```
# one-time setup
.venv/bin/python -m pip install --upgrade unsloth trl transformers datasets accelerate bitsandbytes huggingf
.venv/bin/huggingface-cli download "$BASE" --local-dir "$BASE_LOCAL"

# pipeline (extract → train → merge/convert/install)
.venv/bin/python training/extract_eli_dataset.py --base-model "$BASE_LOCAL" --out training/datasets/eli_voic
.venv/bin/python training/train_lora.py         --base-model "$BASE_LOCAL" --dataset training/datasets/eli_
.venv/bin/python training/merge_and_convert.py   --base-model "$BASE_LOCAL" --adapter "training/runs/$NAME-
# then: load models/$NAME-q4_k_m.gguf in ELI (Model menu / Auto Detect)
```

File map

- `training/extract_eli_dataset.py` — dataset (voice only, persona-dynamic-safe)
- `training/train_lora.py` — QLoRA trainer (parameterised, Qwen3-8B default)
- `training/merge_and_convert.py` — merge → GGUF → quantize → install
- `training/README.md` — quick-start; this doc — the full rationale + correctness
- `training/datasets/`, `training/runs/`, `training/base/` — data, adapters, base weights

Golden rules, one more time

1. Fine-tune **voice + contracts**, never the dynamic persona/memory.
2. **Clean** the dataset; quality over quantity.
3. **Smoke-run**, then watch the **loss**; stop before it memorizes.
4. Base model must have the **context** to hold ELI’s brief (≥16k).
5. **Verify on real prompts** (voice + contracts + dynamic-persona + no-regression), not on loss.